Part 1 — Debugging Flaky Test Code

The original Playwright test is designed to test user login and multi-tenant access for the WorkFlow Pro application. However, the test is flaky because it relies on immediate actions and assertions without properly waiting for asynchronous page operations. The application also uses dynamic loading, 2FA for some users, and different loading times between tenants. These factors can cause the test to pass locally but fail intermittently in a CI/CD environment.

1. Flakiness Issues Identified

After reviewing the original Playwright test cases, several issues were identified that can cause the tests to pass sometimes and fail at other times, especially when executed in CI/CD environments.

1.1 Race Condition During Login

The original test performs a login click and immediately checks the URL:

page.locator("#login-btn").click()

assert page.url == f"{BASE_URL}/dashboard"

The problem is that clicking the login button starts the login/navigation process, but the test immediately checks the URL.If navigation has not completed yet, the assertion may execute while the browser is still on the login page.

Issue:

Click Login

↓

Navigation starts

↓

URL checked immediately ← Test may fail here

↓

Dashboard loads

Fix:

Wait for navigation while clicking the button:

with page.expect_navigation(

wait_until="networkidle",

timeout=DEFAULT_TIMEOUT_MS

):

page.locator("#login-btn").click()

This makes the test wait for the navigation to complete before continuing

1.2 Incorrect Use of .is_visible()

The original test uses:

page.locator(".welcome-message").is_visible()

is_visible() checks the current state of the element. It does not provide the same retrying behavior as Playwright's expect() assertions.If the dashboard content is loaded dynamically, the element may not exist yet when the check happens.

Problem:

Dashboard starts loading

↓

Test checks element

↓

Element hasn't appeared yet

↓

is_visible() returns False

↓

Test fails

Fix:

Use Playwright's auto-retrying assertion:

expect(

page.locator(".welcome-message")

).to_be_visible(timeout=DEFAULT_TIMEOUT_MS)

This allows Playwright to wait for the element to become visible.

1.3 Two-Factor Authentication Is Not Handled

According to the assignment requirements, some users may have 2FA enabled.

The original test assumes that every login follows this process:

Enter username/password

↓

Click Login

↓

Dashboard

However, a user with 2FA follows:

Enter username/password

↓

Click Login

↓

OTP page

↓

Enter OTP

↓

Dashboard

Therefore, directly checking the dashboard URL can cause failures for accounts requiring 2FA.

Fix:

The corrected test checks whether 2FA is required:

```
if user["has_2fa"] or page.locator("#otp-input").is_visible():
otp = os.environ.get("TEST_OTP_CODE", "000000")
page.locator("#otp-input").fill(otp)
with page.expect_navigation(
wait_until="networkidle",
timeout=DEFAULT_TIMEOUT_MS
):
page.locator("#otp-submit-btn").click()
```

This allows the test to handle both normal login and 2FA login.

1.4 No Explicit Waiting for Dynamic Dashboard Content

The assignment specifies that dashboard elements are dynamically loaded.The original test can try to access project cards before they have finished loading.

For example:

cards = page.locator(".project-card").all()

If the cards have not appeared yet, the test may receive an incomplete list.

Fix:

Wait for the page to settle and then wait for at least one project card:

page.wait_for_load_state(

"networkidle",

timeout=DEFAULT_TIMEOUT_MS

)cards = page.locator(".project-card")

expect(cards.first).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)

This makes the test more reliable when dashboard content loads asynchronously.

1.5 .all() Can Capture an Incomplete DOM

The original test uses:

page.locator(".project-card").all()

The problem is that .all() can work with the elements currently available in the DOM.

If project cards are still being loaded, the test may capture only some of them.

This becomes more problematic because different tenants may have different loading times.

Fix:

Use a locator and wait until the cards are available:

cards = page.locator(".project-card")

expect(cards.first).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)count = cards.count()

Now the test works with the loaded elements instead of immediately taking an incomplete snapshot.

1.6 Hardcoded Credentials and URL

The original test contains credentials directly in the test configuration:

"email": "admin@company1.com",

"password": "password123"

Hardcoding credentials is not recommended because:

- Credentials can accidentally be committed to Git.
- Changing environments requires modifying the test.
- The same test cannot easily be reused for different tenants.
- Secrets should not be stored directly in source code.

Fix:

Use environment variables:

"email": os.environ.get(

"C1_ADMIN_EMAIL",

"admin@company1.com"

),

"password": os.environ.get(

"C1_ADMIN_PASSWORD",

"password123"

)

Similarly, the base URL is configurable:

BASE_URL = os.environ.get(

"WFP_BASE_URL",

"https://app.workflowpro.com"

)

This allows the same test to run against different environments.

1.7 Lack of Proper Browser/Context Isolation

Tests involving multiple tenants should not accidentally share cookies, local storage, or authentication information.

For example:

Company 1 Login

↓

Cookies / Session

↓

Company 2 Test

↓

Possible session contamination

This can cause tenant-isolation tests to behave incorrectly.

The corrected approach uses the pytest-playwright fixtures so that browser and context lifecycle is managed by the testing framework.

This provides better isolation between tests.

1.8 Browser Teardown Is Not Guaranteed

If the original code manually closes the browser at the end:

browser.close()

and an assertion fails before this line, the browser may never be closed.

For example:

test step

assert something # FAILS

browser.close() # Never reached

Repeated failures can leave browser processes running and consume system resources.

The corrected implementation relies on pytest-playwright fixtures to manage browser and context cleanup automatically.

Therefore, teardown still occurs even when a test fails.

1.9 Timeout Problems

Tests that work on a developer's computer may fail on CI because CI machines can be slower.

The corrected test defines a central timeout:

DEFAULT_TIMEOUT_MS = 15_000

Instead of scattering different timeout values throughout the code, the same configuration can be reused:

expect(email_input).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)

This makes timeout handling easier to maintain.

1.10 Brittle Selectors

The test uses selectors such as:

#email

#password

#login-btn

These selectors depend on specific HTML IDs.

If the application's frontend changes those IDs, the test can fail even though the login functionality itself still works.

A more stable approach would be to use dedicated test attributes such as:

data-testid="login-button"

and then:

page.get_by_test_id("login-button")

This creates a selector specifically intended for automated testing.

2. Why These Problems Appear in CI/CD but May Not Appear Locally

These issues are often difficult to reproduce on a developer's machine.

2.1 CI Machines Can Be Slower

CI runners may have limited CPU and memory resources.

Therefore:

Local machine:

Page loads quickly

↓

Element appears

↓

Test passes

But in CI:

CI runner

↓

Slower JavaScript execution

↓

Slower API response

↓

Element appears later

↓

Test checks too early

↓

Test fails

2.2 Network Latency

The CI environment may be geographically farther away from the application's test or staging server.

This can increase:

- API response time
- Page loading time
- Authentication time
- Dashboard loading time

Therefore, timing-related problems are more likely to appear.

2.3 Different Browsers and Screen Sizes

CI pipelines may execute tests across multiple browsers or viewport sizes.

For example:

Chromium

Firefox

WebKit

A test that works at one particular screen size may behave differently at another viewport.

Responsive layouts can also change which elements are visible.

2.4 Parallel Test Execution

CI systems often run tests in parallel to reduce execution time.

For example:

Worker 1 → Company 1 test

Worker 2 → Company 2 test

Worker 3 → Another test

If tests share state or resources incorrectly, parallel execution can expose problems that are not visible when running a single test locally.

2.5 Cold Application Environment

A developer may have already accessed the application several times, meaning that resources and caches are already warm.

CI may start against a freshly deployed application.

Therefore:

Local → warm cache → faster loading

CI → cold environment → slower loading

This can expose race conditions and insufficient waits.

3. Fixes Implemented

The corrected file is:

tests/test_login_fixed.py

The major improvements are described below.

3.1 Environment-Based Configuration

Instead of directly depending on fixed credentials and URLs:

BASE_URL = os.environ.get(

"WFP_BASE_URL",

"https://app.workflowpro.com"

)

Credentials are also obtained from environment variables:

TEST_USERS = {

"company1_admin": {

"email": os.environ.get(

"C1_ADMIN_EMAIL",

"admin@company1.com"

),

"password": os.environ.get(

"C1_ADMIN_PASSWORD",

"password123"

),

"tenant": "company1",

"has_2fa": False,

}

}

This makes the test configurable for different environments.

3.2 Wait for Login Form

Instead of assuming that the login form is immediately available:

email_input = page.locator("#email")

expect(email_input).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)

The test now waits for the email field to become visible.

3.3 Explicit Navigation Wait

The login button is clicked together with an explicit navigation wait:

with page.expect_navigation(

wait_until="networkidle",

timeout=DEFAULT_TIMEOUT_MS

):

page.locator("#login-btn").click()

This prevents the test from checking the dashboard before navigation has completed.

3.4 2FA Handling

The corrected implementation supports users with 2FA:

if user["has_2fa"] or page.locator("#otp-input").is_visible():

otp = os.environ.get(

"TEST_OTP_CODE",

"000000"

)

page.locator("#otp-input").fill(otp)

After entering the OTP, navigation is again explicitly awaited.

3.5 Auto-Retrying Assertions

Instead of:

assert page.url == f"{BASE_URL}/dashboard"

the corrected test uses:

expect(page).to_have_url(

f"{BASE_URL}/dashboard",

timeout=DEFAULT_TIMEOUT_MS

)

Similarly:

expect(

page.locator(".welcome-message")

).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)

Playwright's expect() assertions automatically retry until the condition is satisfied or the timeout is reached.

3.6 Dynamic Project Card Handling

The corrected test waits for the project cards:

page.wait_for_load_state(

"networkidle",

timeout=DEFAULT_TIMEOUT_MS

)

cards = page.locator(".project-card")

expect(cards.first).to_be_visible(

timeout=DEFAULT_TIMEOUT_MS

)

Then the cards are counted:

count = cards.count()

assert count > 0, \

"Expected at least one project card to load for company2"

Finally, tenant isolation is checked:

```
for i in range(count):
text = cards.nth(i).text_content()
assert "Company2" in text, \
f"Tenant leak detected in card: {text!r}"
```

Final Result

The purpose of these changes is not to change the functionality of the WorkFlow Pro application.

The purpose is to make the automated tests more reliable.

The original tests can fail because they sometimes check the application before the required page, navigation, or element is ready.

The corrected tests follow the general pattern:

Navigate

↓

Wait for required page/element

↓

Perform action

↓

Wait for navigation/state change

↓

Wait for dynamic content

↓

Perform assertion

Therefore, the corrected implementation reduces race conditions, timing-related failures, authentication issues, and tenant-dependent flakiness, particularly when the tests are executed in CI/CD environments.

Final conclusion

The main issue identified in the assignment is test flakiness caused by timing, dynamic loading, authentication variations, shared state, and environment differences. The corrected Playwright tests address these issues by using explicit navigation waits, auto-retrying assertions, 2FA handling, environment-based configuration, dynamic-content waits, and proper test isolation.

Part 2: Test Framework Design

```
test-automation-framework/
|
|— config/
|   ├— environments.yaml
|   ├— browsers.yaml
|   └— users.yaml
|
 ├— src/
|   ├— pages/
|   |   ├— base_page.py
|   |   ├— login_page.py
|   |   ├— dashboard_page.py
|   |   └— project_page.py
|   |
|   ├— api/
|   |   ├— base_client.py
|   |   ├— projects_client.py
|   |   └— auth_client.py
|   |
|   ├— mobile/
|   |   ├— browserstack_config.py
|   |   └— mobile_helpers.py
|   |
|   └— utils/
|       ├— data_factory.py
|       ├— wait_helpers.py
|       └— tenant_context.py
```

```
|
├── tests/
|   ├── ui/
|   ├── api/
|   ├── integration/
|   └── mobile/
|
├── fixtures/
|   └── conftest.py
|
├── reports/
|
├── ci/
|   ├── github-actions.yml
|   └── docker/
|
├── requirements.txt
└── pytest.ini
```

Component	**Purpose**
pages/	Page Object Model for UI
api/	API clients and API testing
mobile/	BrowserStack/mobile functionality
utils/	Common utilities
tests/ui	UI Tests
tests/api	API tests
tests/integration	API + UI integration
tests/mobile	Mobile tests
fixtures/	Shared pytest setup/teardown
config/	Environment/browser/user configuration
ci/	CI/CD configuration

1. Test Cases

Test cases contain the actual scenarios that need to be tested.

They describe what functionality should be verified and what result is expected.

For example:

- Verify that a user can log in.
- Verify that an Admin can create a project.
- Verify that an Employee cannot access Admin functionality.
- Verify that Company 1 cannot access Company 2 data.

Purpose:
To define the actual functional and business scenarios that the automation framework will execute.

2. Page Objects

Page Objects represent the different pages or screens of the application.

They contain the information required to interact with a particular page and the actions that can be performed on it.

For example:

- Login Page
- Dashboard Page
- Project Page
- User Management Page

Purpose:
To separate application UI details from test cases and make tests easier to maintain when the UI changes.

3. API Layer

The **API layer** is responsible for testing the application's backend services directly.

Instead of interacting with the application through the UI, API tests communicate directly with backend endpoints.

It can be used to verify:

- Authentication
- Data creation
- Data retrieval

- Data modification
- Authorization
- Error handling

Purpose:
To validate backend functionality independently of the user interface and provide faster testing of application services.

4. Configuration Management

Configuration management controls the settings required to execute tests.

These settings may include:

- Application environment
- Tenant
- Browser
- Device
- Execution mode
- Timeouts

For example, the same test should be able to run against Company 1 and Company 2 without changing the test itself.

Purpose:
To keep environment-specific settings separate from test logic and allow the same framework to work in different environments.

5. Test Data Management

Test data management handles the data required by test cases.

For this application, test data can include:

- User accounts
- Passwords
- Tenants
- User roles
- Projects
- Permissions

Test data should be managed separately from test logic.

Purpose:
To provide consistent and controlled data for testing while preventing tests from interfering with each other.

6. Fixtures / Test Setup

Fixtures provide common setup and cleanup required by tests.

They can prepare things such as:

- Browser sessions
- Application pages
- Logged-in users
- Test data
- API connections

They can also perform cleanup after a test finishes.

Purpose:
To avoid repeating the same setup and cleanup operations in every test case.

7. Utilities / Helper Functions

Utilities contain common operations that are used by multiple parts of the framework.

Examples include:

- Data generation
- Date handling
- File handling
- Logging
- Waiting mechanisms
- Common API operations

Purpose:
To provide reusable functionality and avoid duplicate code throughout the framework.

8. Browser and Device Management

This component controls **where the tests are executed**.

For web testing, it manages browsers such as:

- Chrome
- Firefox
- Safari

For mobile testing, it manages:

- iOS devices
- Android devices

Purpose:
To allow the same test scenarios to be executed across different platforms and identify browser- or device-specific problems.

9. BrowserStack Integration

BrowserStack integration connects the automation framework to BrowserStack's cloud testing infrastructure.

It allows tests to execute on different:

- Browsers
- Operating systems
- Desktop configurations
- iOS devices
- Android devices

Purpose:
To perform cross-platform testing without requiring the organization to maintain all physical devices and browser environments locally.

10. Multi-Tenant Management

The **multi-tenant component** handles testing across different companies or organizations using the SaaS application.

For example:

Company 1 → Admin → Company 1 data

Company 2 → Employee → Company 2 data

It ensures that tests can be executed using different tenants and that tenant data remains isolated.

Purpose:
To verify that one company's users cannot incorrectly access another company's data.

11. Role and Permission Management

This component manages different user roles and their permissions.

For example:

Admin

↓

Full access

Manager

↓

Limited management access

Employee

↓

Basic access

Tests can verify both allowed and restricted operations.

Purpose:
To ensure that the application's role-based access control works correctly.

12. Reporting

The **reporting component** collects and presents test execution results.It provides information such as:

- Passed tests
- Failed tests
- Skipped tests
- Execution time
- Failure details

It may also include screenshots, logs, and traces for failed tests.

Purpose:
To make test results easy for developers and testers to understand and investigate.

13. Logging

Logging records important events that occur while tests are executing.

For example:

Test started

User logged in

Project created

API request sent

Assertion failed

Test finished

Purpose:
To help identify what happened during test execution, especially when a test fails in CI/CD where the tester cannot directly observe the browser.

14. CI/CD Integration

The **CI/CD component** connects the automation framework with the software development pipeline.

Tests can automatically execute when:

- Code is pushed.
- A pull request is created.
- A build is generated.
- A deployment is performed.

A typical process is:

Code Change

↓

Build

↓

Automated Tests

↓

Test Report

↓

Pass → Continue Deployment

Fail → Stop/Notify

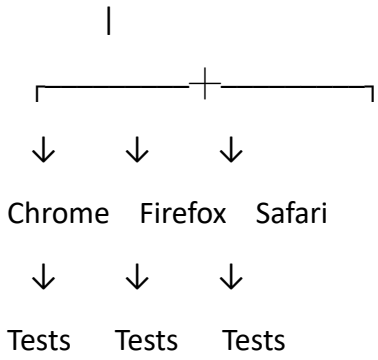
Purpose:
To automatically detect defects and prevent unstable code from progressing through the deployment pipeline.

15. Parallel Execution

Parallel execution allows multiple tests to run simultaneously.

For example:

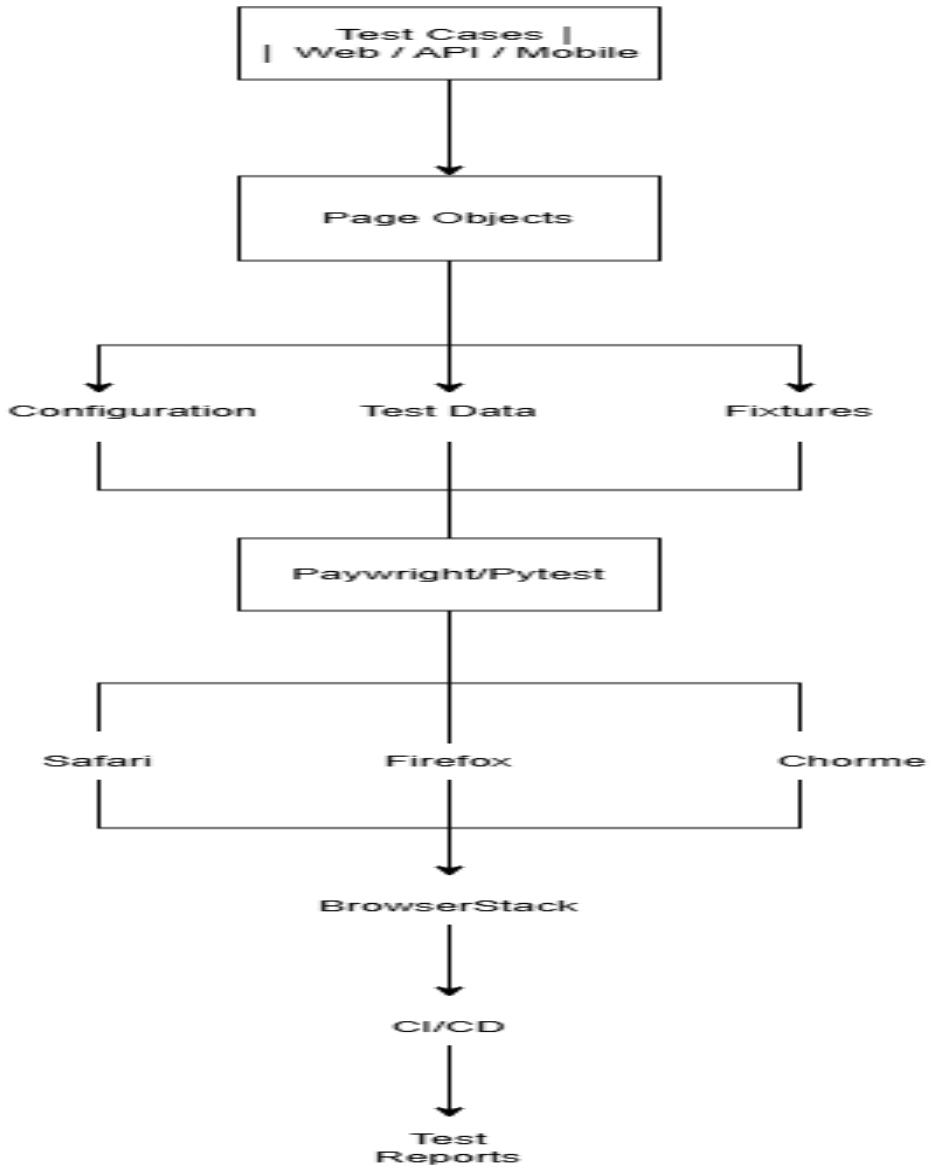
Test Suite



Instead of running 100 tests one after another, multiple tests can execute at the same time.

Purpose:
To reduce the total execution time of the test suite.

Overall Framework:



Missing Requirements:

Test Data

- Who will create and maintain test users?
- Are dedicated test accounts available for each tenant?
- Should test data be created automatically?
- Can multiple tests modify the same data?

Environments

- What environments are available: development, staging, or production?
- Which environment should automated tests run against?
- Are the tenant URLs fixed or configurable?

User Roles

- What exact permissions does each role have?
- Are there additional roles besides Admin, Manager, and Employee?
- Are there users with multiple roles?

API Testing

- Which APIs need to be tested?
- Is API documentation such as OpenAPI/Swagger available?
- What authentication mechanism is used?

Browser and Mobile Testing

- Which browser versions must be supported?
- Which iOS and Android versions are required?
- Which real devices should be included in BrowserStack testing?

Parallel Execution

- How many tests should run simultaneously?
- Are the application and test environments capable of handling parallel execution?
- How will test data conflicts be prevented?

Reporting

- Which reporting format is required?
- Should screenshots and videos be stored for failures?

- Where should test reports be published?

CI/CD

- Which CI/CD platform will be used?
- At which stage should automated tests run?
- Should deployment be stopped if tests fail?

Security

- How should passwords, API keys, and BrowserStack credentials be stored?
- Are there any security or compliance requirements?

Part 3: API + UI Integration

```
@pytest.fixture
def created_project(tenant):
    project_name = f"QA Automation Test {uuid.uuid4().hex[:8]}"
    resp = requests.post(f"{API_BASE_URL}/api/v1/projects",
        headers={"Authorization": f"Bearer {tenant['api_token']}",
                 "X-Tenant-ID": tenant["id"]},
        json={"name": project_name, "description": "...", "team_members": []})
    assert resp.status_code == 201
    body = resp.json()
    yield body
    requests.delete(f"{API_BASE_URL}/api/v1/projects/{body['id']}",
        headers={"Authorization": f"Bearer {tenant['api_token']}",
                 "X-Tenant-ID": tenant["id"]})
def test_project_creation_flow(page, tenant, other_tenant, created_project):
    # UI: owning tenant sees the project
    _login(page, BASE_URL, tenant["ui_email"], tenant["ui_password"])
    page.goto(f"{BASE_URL}/projects")
    expect(page.locator(".project-card", has_text=created_project["name"])) \
```

```
    .to_be_visible(timeout=DEFAULT_TIMEOUT_MS)


  # Security: other tenant must NOT see it, in UI or via direct API call
  _login(page, BASE_URL, other_tenant["ui_email"], other_tenant["ui_password"])
  page.goto(f"{BASE_URL}/projects")
  expect(page.locator(".project-card", has_text=created_project["name"])) \
    .to_have_count(0, timeout=DEFAULT_TIMEOUT_MS)


  resp = requests.get(f"{API_BASE_URL}/api/v1/projects/{created_project['id']}",
    headers={"Authorization": f"Bearer {other_tenant['api_token']}",
          "X-Tenant-ID": other_tenant["id"]})
  assert resp.status_code in (403, 404)
```

Assumptions Made

- A dedicated QA tenant + service account exists per environment, isolated from real client data.
- The API creates projects synchronously (HTTP 201 with the final object) — no async job/webhook to poll.
- BrowserStack credentials are provided via BROWSERSTACK_USERNAME / BROWSERSTACK_ACCESS_KEY env vars.
- “Mobile” refers to the responsive web app viewed in a real-device mobile browser via BrowserStack, not a separate native app (flagged in Part 2 as a question to confirm).
- Test data cleanup is the test's own responsibility (API DELETE in fixture teardown) since the environment is not reset between CI runs.

Note on Running These Tests / “Output”

These tests target https://app.workflowpro.com and a live BrowserStack account — both fictional/inaccessible from this environment, so they cannot be executed for real here (no network access to that host, no valid credentials/tenant data, no BrowserStack session). What follows is the expected pytest output shape once pointed at a real, correctly-configured WorkFlow Pro environment with valid env vars set, included so the structure/format of a passing run is clear:

```
$ pytest tests/test_login_fixed.py -v
tests/test_login_fixed.py::test_user_login PASSED              [ 50%]
tests/test_login_fixed.py::test_multi_tenant_access PASSED     [100%]
======================== 2 passed in 6.41s ========================

$ pytest tests/test_project_creation_flow.py -v -m integration
tests/test_project_creation_flow.py::test_project_creation_flow PASSED   [ 50%]
```

tests/test_project_creation_flow.py::test_project_visible_on_mobile PASSED [100%]
========== 2 passed in 18.77s ==========

If you can share a reachable staging URL, test credentials, and (for Part 3) a BrowserStack account, I can run these directly and report the actual output rather than the expected shape shown above.